LAPORAN LANJUTAN TUGAS MANDIRI
D.2, D.3, dan D.4 -- Modul 9: Transactions & Concurrency

Pelengkap Berita Acara Perbaikan (D.1) pada berkas Tugas9_434241021_D1.pdf

Program Studi	D4 Teknik Informatika -- Fakultas Vokasi, Universitas Airlangga
NIM	434241021
Mata Kuliah	Praktikum Pemrograman Backend Lanjut (2 SKS) -- Semester 5
Folder proyek	latihan-fiber/MODUL_9 (commit D.2: d3d6bee)
Isi laporan ini	D.2 Entity students (Pembimbing TA), D.3 Bukti Uji Beban, D.4 Analisis Singkat

6. Tugas Mandiri D.2 -- Pembimbing Tugas Akhir untuk Entity students

Seluruh pola pertemuan ini diterapkan pada entity students: migration berisi tabel supervisors dan supervisions, endpoint penetapan dengan UPDATE bersyarat untuk kuota, kunci baris students untuk mencegah satu mahasiswa ditetapkan ke dua pembimbing serentak, endpoint pergantian yang atomik dan bebas deadlock, PATCH kuota berbasis optimistic locking, serta repository yang berpola DBTX/WithTx dengan transaction yang dibuka lewat TxManager.

6.1 Desain data dan keputusan

Keputusan	Lokasi	Alasan
UNIQUE(student_id) pada supervisions	migrations/007_supervisors.sql:22	Aturan 'seorang mahasiswa hanya boleh memiliki SATU pembimbing' dijaga level database; dua penetapan serentak tidak mungkin sama-sama berhasil.
CHECK supervised_count >= 0 dan <= quota	007_supervisors.sql:12-13	Jaring pengaman terakhir; bila kode aplikasi benar, constraint ini tidak pernah terpicu.
LockStudent ... FOR UPDATE	app/repository/supervisor_repository.go:101-112	Gembok per mahasiswa: seluruh operasi (assign/switch) milik mahasiswa yang sama antre satu per satu.
LockSupervisors ... ORDER BY id FOR UPDATE	supervisor_repository.go:116-139	Semua transaksi mengambil kunci pembimbing dalam urutan id menaik -- bebas deadlock.
UPDATE bersyarat supervised_count < quota	supervisor_repository.go:144-147	Pengambilan kursi kuota berlangsung atomik (Cara 1, Bagian A.4).
version = version + 1 WHERE version = $3	supervisor_repository.go:201-204	Optimistic locking: penyimpan dengan versi usang ditolak 409, bukan menimpa (Cara 3, A.4).

6.2 Migration 007 -- tabel, constraint, permission, dan seed

`migrations/007_supervisors.sql`

```
-- Migration 007: Dosen Pembimbing dan Penetapan Tugas Akhir
-- Kasus: Penetapan Dosen Pembimbing untuk Entity students (Tugas Mandiri D.2)

CREATE TABLE IF NOT EXISTS supervisors (
    id SERIAL PRIMARY KEY,
    name VARCHAR(120) NOT NULL,
    quota INTEGER NOT NULL CHECK (quota > 0),
    supervised_count INTEGER NOT NULL DEFAULT 0,
    version INTEGER NOT NULL DEFAULT 1,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    -- Jaring pengaman terakhir: jumlah bimbingan tidak boleh melebihi kuota
    CONSTRAINT supervisors_supervised_within_quota
        CHECK (supervised_count >= 0 AND supervised_count <= quota)
);

CREATE TABLE IF NOT EXISTS supervisions (
    id BIGSERIAL PRIMARY KEY,
    student_id INTEGER NOT NULL REFERENCES students(id) ON DELETE CASCADE,
    supervisor_id INTEGER NOT NULL REFERENCES supervisors(id) ON DELETE CASCADE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    -- Seorang mahasiswa hanya boleh memiliki SATU pembimbing di level database:
    CONSTRAINT supervisions_student_unique UNIQUE (student_id)
);

CREATE INDEX IF NOT EXISTS supervisions_supervisor_idx ON supervisions (supervisor_id);

-- Permission baru: supervision:manage
INSERT INTO permissions (name, description) VALUES
('supervision:manage', 'Menetapkan dan mengelola dosen pembimbing tugas akhir')
ON CONFLICT (name) DO NOTHING;

-- Berikan permission supervision:manage ke role admin dan staff
```

```
INSERT INTO role_permissions (role_name, permission_name) VALUES
('admin', 'supervision:manage'),
('staff', 'supervision:manage')
ON CONFLICT DO NOTHING;

-- Seed beberapa data pembimbing
-- ID 1: kuota 3 (digunakan untuk uji beban D.3)
-- ID 2: kuota 5 (digunakan untuk uji switch pembimbing)
-- ID 3: kuota 10
INSERT INTO supervisors (id, name, quota) VALUES
(1, 'Dr. Ir. Budi Santoso, M.Kom.', 3),
(2, 'Dr. Eng. Siti Aminah, S.T., M.T.', 5),
(3, 'Prof. Dr. Agus Raharjo, M.Cs.', 10)
ON CONFLICT (id) DO NOTHING;

-- Seed 60 data mahasiswa uji untuk beban penetapan serentak D.3
INSERT INTO students (nim, name, grade, is_active)
SELECT
    '434241021' || LPAD(g::TEXT, 3, '0'),
    'Mahasiswa TA ' || LPAD(g::TEXT, 3, '0'),
    'A',
    TRUE
FROM generate_series(1, 60) AS g
ON CONFLICT (nim) DO NOTHING;
```

6.3 Repository -- pola DBTX, WithTx, kunci, dan UPDATE bersyarat

Sama seperti CourseRepository, struct repository menyimpan kemampuan query generik DBTX, bukan pool; method WithTx menghasilkan salinan repository yang seluruh query-nya berjalan di dalam transaction yang diberikan TxManager. Interface penuh dan daftar error domain:

app/repository/supervisor_repository.go (interface + error domain)

```
type SupervisorRepository interface {
    WithTx(tx DBTX) SupervisorRepository
    FindAll(ctx context.Context) ([]model.Supervisor, error)
    FindByID(ctx context.Context, id int) (model.Supervisor, error)
    FindByStudentID(ctx context.Context, studentID int) (model.Supervision, error)
    LockStudent(ctx context.Context, studentID int) error
    LockSupervisors(ctx context.Context, ids ...int) error
    IncrementSupervised(ctx context.Context, supervisorID int) (model.Supervisor, error)
    DecrementSupervised(ctx context.Context, supervisorID int) (model.Supervisor, error)
    InsertSupervision(ctx context.Context, studentID, supervisorID int) error
    DeleteSupervision(ctx context.Context, studentID, supervisorID int) error
    UpdateQuota(ctx context.Context, id, quota, version int) (model.Supervisor, error)
}

var (
    ErrSupervisorFull            = errors.New("kuota pembimbing sudah penuh")
    ErrAlreadySupervised         = errors.New("mahasiswa sudah memiliki pembimbing")
    ErrNotSupervised             = errors.New("mahasiswa tidak dibimbing oleh dosen tersebut")
    ErrSupervisorVersionConflict = errors.New("data pembimbing sudah diubah pihak lain")
    ErrQuotaBelowSupervised      = errors.New("kuota tidak boleh lebih kecil dari jumlah mahasiswa
      bimbingan saat ini")
)

const supervisorColumns = `id, name, quota, supervised_count, version, created_at`
```

LockStudent mengunci baris tabel students sebagai 'gembok', LockSupervisors mengunci kandidat pembimbing dalam urutan id menaik agar lingkaran tunggu (deadlock) tak mungkin terbentuk:

LockStudent + LockSupervisors (pessimistic locking)

```
// LockStudent mengunci baris students untuk mencegah satu mahasiswa ditetapkan
// ke dua pembimbing secara serentak (Pessimistic Locking).
func (r *supervisorPostgresRepository) LockStudent(ctx context.Context, studentID int) error {
    var id int
    err := r.db.QueryRow(ctx,
        "SELECT id FROM students WHERE id = $1 FOR UPDATE", studentID).Scan(&id)
    if err != nil {
        if errors.Is(err, pgx.ErrNoRows) {
            return ErrNotFound
        }
        return fmt.Errorf("mengunci data mahasiswa: %w", err)
    }
    return nil
}

// LockSupervisors mengunci beberapa pembimbing sekaligus dalam urutan ID menaik (ORDER BY id)
// untuk mencegah deadlock saat terjadi pergantian pembimbing berlawanan arah secara serentak.
func (r *supervisorPostgresRepository) LockSupervisors(ctx context.Context, ids ...int) error {
    arr := make([]int32, len(ids))
    for i, id := range ids {
        arr[i] = int32(id)
    }
    rows, err := r.db.Query(ctx,
        "SELECT id FROM supervisors WHERE id = ANY($1) ORDER BY id FOR UPDATE", arr)
    if err != nil {
        return fmt.Errorf("mengunci pembimbing: %w", err)
    }
    defer rows.Close()

    found := 0
    for rows.Next() {
        found++
```

```
    }
    if err := rows.Err(); err != nil {
        return fmt.Errorf("mengunci pembimbing: %w", err)
    }
    if found != len(ids) {
        return ErrNotFound
    }
    return nil
}
```

IncrementSupervised adalah jantung kuota: satu UPDATE bersyarat yang mengembalikan 0 baris saat kuota penuh, lalu dibedakan antara kemungkinan 'tidak ada dosen' dan 'kuota penuh'. DeleteSupervision MEMERIKSA jumlah baris terhapus agar pengurangan kuota fiktif tidak terjadi:

IncrementSupervised, DecrementSupervised, InsertSupervision, DeleteSupervision

```
// IncrementSupervised menambah jumlah bimbingan secara ATOMIK dengan UPDATE bersyarat.
// Kuota diperiksa dan diperbarui dalam satu query (supervised_count < quota).
func (r *supervisorPostgresRepository) IncrementSupervised(ctx context.Context, supervisorID int)
      (model.Supervisor, error) {
    s, err := scanSupervisor(r.db.QueryRow(ctx,
        `UPDATE supervisors SET supervised_count = supervised_count + 1
        WHERE id = $1 AND supervised_count < quota
        RETURNING `+supervisorColumns, supervisorID))
    if err == nil {
        return s, nil
    }
    if !errors.Is(err, pgx.ErrNoRows) {
        return model.Supervisor{}, fmt.Errorf("menambah bimbingan: %w", err)
    }
    if _, findErr := r.FindByID(ctx, supervisorID); findErr != nil {
        return model.Supervisor{}, findErr
    }
    return model.Supervisor{}, ErrSupervisorFull
}

func (r *supervisorPostgresRepository) DecrementSupervised(ctx context.Context, supervisorID int)
      (model.Supervisor, error) {
    s, err := scanSupervisor(r.db.QueryRow(ctx,
        `UPDATE supervisors SET supervised_count = supervised_count - 1
        WHERE id = $1
        RETURNING `+supervisorColumns, supervisorID))
    if err != nil {
        if errors.Is(err, pgx.ErrNoRows) {
            return model.Supervisor{}, ErrNotFound
        }
        return model.Supervisor{}, fmt.Errorf("mengurangi bimbingan: %w", err)
    }
```

```
    return s, nil
}

func (r *supervisorPostgresRepository) InsertSupervision(ctx context.Context, studentID, supervisorID
      int) error {
    _, err := r.db.Exec(ctx,
        "INSERT INTO supervisions (student_id, supervisor_id) VALUES ($1, $2)", studentID,
      supervisorID)
    if err != nil {
        if pgErrorCode(err) == codeUniqueViolation {
            return ErrAlreadySupervised
        }
        return fmt.Errorf("mencatat bimbingan: %w", err)
    }
    return nil
}

func (r *supervisorPostgresRepository) DeleteSupervision(ctx context.Context, studentID, supervisorID
      int) error {
    tag, err := r.db.Exec(ctx,
        "DELETE FROM supervisions WHERE student_id = $1 AND supervisor_id = $2", studentID,
      supervisorID)
    if err != nil {
        return fmt.Errorf("menghapus bimbingan: %w", err)
    }
    if tag.RowsAffected() == 0 {
        return ErrNotSupervised
    }
    return nil
}
```

UpdateQuota menerapkan optimistic locking: tanpa kunci, versi menentukan penerimaan:

UpdateQuota (optimistic locking)

```
// UpdateQuota menerapkan Optimistic Locking dengan menaikkan kolom version = version + 1
// dan memeriksa bahwa versi di database masih sama dengan yang dilihat client.
func (r *supervisorPostgresRepository) UpdateQuota(ctx context.Context, id, quota, version int)
      (model.Supervisor, error) {
    s, err := scanSupervisor(r.db.QueryRow(ctx,
        `UPDATE supervisors SET quota = $1, version = version + 1
        WHERE id = $2 AND version = $3 AND $1 >= supervised_count
        RETURNING `+supervisorColumns, quota, id, version))
    if err == nil {
        return s, nil
    }
    if !errors.Is(err, pgx.ErrNoRows) {
        return model.Supervisor{}, fmt.Errorf("mengubah kuota pembimbing: %w", err)
    }
    current, findErr := r.FindByID(ctx, id)
    if findErr != nil {
        return model.Supervisor{}, findErr
    }
    if current.Version != version {
        return model.Supervisor{}, ErrSupervisorVersionConflict
    }
    return model.Supervisor{}, ErrQuotaBelowSupervised
}
```

6.4 Service -- Assign, Switch, dan UpdateQuota

Hanya service yang mengetahui langkah mana yang harus berhasil bersama (A.7), karenanya transaction dibuka di sini lewat TxManager. Assign: kunci mahasiswa -- cek bimbingan kosong -- ambil kuota secara atomik -- catat; kegagalan kapan pun membatalkan seluruh langkah:

app/service/supervisor_service.go -- Assign

```
// ---------- POST /supervisors/:id/assign ----------
// Penetapan pembimbing tugas akhir:
// Menggunakan UPDATE bersyarat untuk kuota dan kunci baris students
// untuk mencegah satu mahasiswa ditetapkan ke dua pembimbing secara serentak.
func (s *SupervisorService) Assign(c *fiber.Ctx) error {
    ctx, cancel := helper.RequestContext(c)
    defer cancel()

    supervisorID, valid := helper.ParamID(c)
    if !valid {
        return helper.BadRequest("id harus berupa angka positif")
    }

    var req model.AssignSupervisorRequest
    if err := c.BodyParser(&req); err != nil {
        return helper.BadRequest("body harus berupa JSON yang valid")
    }
    if errs := helper.ValidateStruct(req); errs != nil {
        return helper.Validation(errs)
    }

    var assigned model.Supervisor
    err := s.tx.WithTx(ctx, func(tx repository.DBTX) error {
        repo := s.supervisors.WithTx(tx)

        // 1. Kunci baris mahasiswa (Pessimistic Locking)
        // Mencegah mahasiswa yang sama ditetapkan ke dua pembimbing secara serentak
        if err := repo.LockStudent(ctx, req.StudentID); err != nil {
            return err
        }

        // 2. Periksa apakah mahasiswa sudah memiliki pembimbing
```

```
        existing, err := repo.FindByStudentID(ctx, req.StudentID)
        if err == nil && existing.ID != 0 {
            return repository.ErrAlreadySupervised
        }
        if err != nil && !errors.Is(err, repository.ErrNotFound) {
            return err
        }

        // 3. Ambil satu kuota pembimbing secara atomik (UPDATE bersyarat)
        assigned, err = repo.IncrementSupervised(ctx, supervisorID)
        if err != nil {
            return err
        }

        // 4. Catat penetapan pembimbing
        return repo.InsertSupervision(ctx, req.StudentID, supervisorID)
    })
    if err != nil {
        return translateSupervisorError(err)
    }

    return helper.Created(c, "berhasil menetapkan dosen pembimbing", assigned,
        "/api/v1/supervisors/"+strconv.Itoa(supervisorID))
}
```

Switch tetap atomik: bila pembimbing tujuan penuh, error pada langkah 6 membatalkan juga penghapusan dan pengurangan pada langkah 4-5; mahasiswa TETAP dengan pembimbing lamanya. Penguncian dua pembimbing lebih dulu dalam urutan id menaik menutup kemungkinan deadlock:

app/service/supervisor_service.go -- Switch

```
// ---------- POST /supervisions/switch ----------
// Pergantian dosen pembimbing tugas akhir:
// Wajib atomik — bila pembimbing baru penuh, mahasiswa tetap dengan pembimbing lama —
// dan wajib bebas deadlock (penguncian seragam dengan urutan id menaik).
func (s *SupervisorService) Switch(c *fiber.Ctx) error {
    ctx, cancel := helper.RequestContext(c)
    defer cancel()

    var req model.SwitchSupervisorRequest
    if err := c.BodyParser(&req); err != nil {
        return helper.BadRequest("body harus berupa JSON yang valid")
    }
    if errs := helper.ValidateStruct(req); errs != nil {
        return helper.Validation(errs)
    }
    if req.FromSupervisorID == req.ToSupervisorID {
        return helper.Validation(map[string]string{
            "to_supervisor_id": "tidak boleh sama dengan from_supervisor_id",
        })
    }

    var joined model.Supervisor
    err := s.tx.WithTx(ctx, func(tx repository.DBTX) error {
        repo := s.supervisors.WithTx(tx)

        // 1. Kunci baris mahasiswa
        if err := repo.LockStudent(ctx, req.StudentID); err != nil {
            return err
        }

        // 2. Kunci KEDUA pembimbing secara seragam dalam urutan ID menaik (Bebas Deadlock)
        if err := repo.LockSupervisors(ctx, req.FromSupervisorID, req.ToSupervisorID); err != nil {
```

```
            return err
        }

        // 3. Verifikasi bahwa mahasiswa saat ini memang dibimbing oleh FromSupervisorID
        existing, err := repo.FindByStudentID(ctx, req.StudentID)
        if err != nil {
            return repository.ErrNotSupervised
        }
        if existing.SupervisorID != req.FromSupervisorID {
            return repository.ErrNotSupervised
        }

        // 4. Hapus bimbingan lama
        if err := repo.DeleteSupervision(ctx, req.StudentID, req.FromSupervisorID); err != nil {
            return err
        }

        // 5. Kurangi bimbingan pembimbing lama
        if _, err := repo.DecrementSupervised(ctx, req.FromSupervisorID); err != nil {
            return err
        }

        // 6. Tambah bimbingan pembimbing baru secara atomik (UPDATE bersyarat)
        // Bila pembimbing baru penuh, error membatalkan langkah 4 dan 5 (Rollback),
        // sehingga mahasiswa tetap dengan pembimbing lama
        joined, err = repo.IncrementSupervised(ctx, req.ToSupervisorID)
        if err != nil {
            return err
        }

        // 7. Catat bimbingan baru
        return repo.InsertSupervision(ctx, req.StudentID, req.ToSupervisorID)
```

```
    })
    if err != nil {
        return translateSupervisorError(err)
    }

    return helper.Success(c, fiber.StatusOK, "berhasil mengganti dosen pembimbing", joined)
}
```

UpdateQuota (tanpa WithTx: satu pernyataan UPDATE sudah atomik) + pemetaan error

```
// ---------- PATCH /supervisors/:id ----------
// Mengubah kuota pembimbing dengan Optimistic Locking
func (s *SupervisorService) UpdateQuota(c *fiber.Ctx) error {
    ctx, cancel := helper.RequestContext(c)
    defer cancel()

    id, valid := helper.ParamID(c)
    if !valid {
        return helper.BadRequest("id harus berupa angka positif")
    }

    var req model.UpdateSupervisorQuotaRequest
    if err := c.BodyParser(&req); err != nil {
        return helper.BadRequest("body harus berupa JSON yang valid")
    }
    if errs := helper.ValidateStruct(req); errs != nil {
        return helper.Validation(errs)
    }

    updated, err := s.supervisors.UpdateQuota(ctx, id, req.Quota, req.Version)
    if err != nil {
        return translateSupervisorError(err)
    }

    return helper.Success(c, fiber.StatusOK, "kuota pembimbing berhasil diubah", updated)
}

func translateSupervisorError(err error) error {
    switch {
    case errors.Is(err, repository.ErrNotFound):
        return helper.NotFound("data tidak ditemukan")
    case errors.Is(err, repository.ErrSupervisorFull):
```

```
            return helper.NewError(fiber.StatusConflict, "SUPERVISOR_FULL", "kuota pembimbing sudah
          penuh")
        case errors.Is(err, repository.ErrAlreadySupervised):
            return helper.NewError(fiber.StatusConflict, "ALREADY_SUPERVISED", "mahasiswa sudah memiliki
          pembimbing")
        case errors.Is(err, repository.ErrNotSupervised):
            return helper.NewError(fiber.StatusConflict, "NOT_SUPERVISED", "mahasiswa tidak dibimbing
          oleh dosen tersebut")
        case errors.Is(err, repository.ErrSupervisorVersionConflict):
            return helper.NewError(fiber.StatusConflict, "VERSION_CONFLICT", "data pembimbing sudah
          diubah pihak lain, muat ulang lalu coba lagi")
        case errors.Is(err, repository.ErrQuotaBelowSupervised):
            return helper.NewError(fiber.StatusConflict, "QUOTA_BELOW_SUPERVISED", "kuota tidak boleh
          lebih kecil dari jumlah mahasiswa bimbingan saat ini")
        default:
            return helper.Internal(err)
        }
    }
```

6.5 Route dan permission

route/route.go -- grup supervisors dan supervisions

```
    // --- pembimbing tugas akhir (Tugas Mandiri D.2) ---
    supervisors := api.Group("/supervisors",
        middleware.RequireJSON,
        middleware.RequireAuth(deps.JWT))
    supervisors.Get("/", deps.SupervisorService.List)
    supervisors.Get("/:id", deps.SupervisorService.Get)
    supervisors.Post("/:id/assign",
        middleware.RequirePermission(perms, "supervision:manage"),
        deps.SupervisorService.Assign)
    supervisors.Patch("/:id",
        middleware.RequirePermission(perms, "supervision:manage"),
        deps.SupervisorService.UpdateQuota)

    supervisions := api.Group("/supervisions",
        middleware.RequireJSON,
        middleware.RequireAuth(deps.JWT))
    supervisions.Post("/switch",
        middleware.RequirePermission(perms, "supervision:manage"),
        deps.SupervisorService.Switch)
```

Endpoint PATCH/assign/switch dijaga permission supervision:manage (middleware RequirePermission); role user tanpa permission ditolak 403 -- dibuktikan pada tahap 7 dari Uji Fungsional bagian 7.3. Sesuai pengingat modul, permission dimuat saat aplikasi menyala: server dijalankan ulang setelah migration.

Endpoint	Guard	Sifat konkurensi	409 / 403 yang mungkin
POST /supervisors/:id/assign	supervision:manage	LockStudent + UPDATE bersyarat kuota + UNIQUE(student_id)	SUPERVISOR_FULL, ALREADY_SUPERVISED, 403
POST /supervisions/switch	supervision:manage	LockStudent + LockSupervisors (urutan id); delete+insert atomik	SUPERVISOR_FULL, NOT_SUPERVISED, 403
PATCH /supervisors/:id	supervision:manage	Optimistic locking kolom version	VERSION_CONFLICT, QUOTA_BELOW_SUPERVISED, 403
GET /supervisors, /supervisors/:id	telah terautentikasi	Hanya baca	-

7. Tugas Mandiri D.3 -- Bukti Uji Beban

Alat uji beban dibuat dengan pola yang sama dan menyerang endpoint supervisors (tools/loadtest_supervisors/main.go, mode assign / assign_broken / switch). Inti serentaknya identik dengan alat modul: fungsi fire menjalankan n goroutine yang semuanya antre di channel start (gerbang baru dibuka setelah SELURUH goroutine siap), sync.WaitGroup menunggu semuanya, dan channel results dengan kapasitas n menampung hasil tanpa tahanan. Alat tersebut juga membuat token sendiri dari JWT_SECRET untuk user admin yang memegang permission supervision:manage, sehingga tidak bergantung pada login.

tools/loadtest_supervisors/main.go -- fungsi gerbang fire()

```
func fire(n int, job func(i int) string) map[string]int {
    var wg sync.WaitGroup
    start := make(chan struct{})

    results := make(chan string, n)
    for i := 0; i < n; i++ {
        wg.Add(1)
        go func(i int, out chan<- string) {
            defer wg.Done()
            <-start
            out <- job(i)
        }(i, results)
    }
    close(start) // buka gerbang: semua goroutine berangkat serentak
    wg.Wait()
    close(results)

    tally := map[string]int{}
    for r := range results {
        tally[r]++
    }
    return tally
}
```

Server uji dijalankan dari kode Modul_9 sendiri (go build -o server_ok . lalu APP_PORT=3099 ./server_ok), dan SEBELUM SETIAP uji beban database dikembalikan ke keadaan awal dengan reset_supervisors.sql. Setiap angka percobaan di bawah ini diambil dan diverifikasi database (psql) langsung pasca run.

Perintah percobaan

```
psql -U postgres -d praktikum_backend -f reset_supervisors.sql   # reset sebelum SETIAP uji
go run ./tools/loadtest_supervisors -mode assign        # diulang 3 kali
go run ./tools/loadtest_supervisors -mode assign_broken # SATU percobaan pembanding
go run ./tools/loadtest_supervisors -mode switch        # 20 pasang berlawanan arah
# verifikasi pasca run:
SELECT id, quota, supervised_count, (SELECT COUNT(*) FROM supervisions v
       WHERE v.supervisor_id = s.id) AS baris
FROM supervisors s WHERE id = 1;
SELECT deadlocks FROM pg_stat_database WHERE datname = 'praktikum_backend';
```

7.1 Tiga percobaan: 30 penetapan serentak ke pembimbing berkuota 3

Percobaan 1

```
30 mahasiswa berebut pembimbing #1 (kuota 3) secara serentak...

=== assign ===
  201                               3
  409 SUPERVISOR_FULL               27
  TOTAL                             30
waktu total : 48.232544ms
db: kuota=3 supervised_count=3 baris=3
deadlocks=0
```

Percobaan 2

```
30 mahasiswa berebut pembimbing #1 (kuota 3) secara serentak...

=== assign ===
  201                               3
  409 SUPERVISOR_FULL               27
  TOTAL                             30
waktu total : 21.630451ms
db: kuota=3 supervised_count=3 baris=3
deadlocks=0
```

Percobaan 3

```
30 mahasiswa berebut pembimbing #1 (kuota 3) secara serentak...

=== assign ===
  201                               3
  409 SUPERVISOR_FULL               27
  TOTAL                             30
waktu total : 23.173245ms
db: kuota=3 supervised_count=3 baris=3
deadlocks=0
```

Ketiga percobaan menghasilkan jumlah yang sama -- tidak satu pun mahasiswa melebihi kuota -- dan pg_stat_database.deadlocks tidak bertambah pada seluruh percobaan:

Run	201 OK	409 SUPERVISOR_FULL	Total	Waktu	DB pasca run	Deadlock
1	3	27	30	48,2 ms	kuota=3, count=3, baris=3 (KONSISTEN)	0
2	3	27	30	21,6 ms	kuota=3, count=3, baris=3 (KONSISTEN)	0
3	3	27	30	23,2 ms	kuota=3, count=3, baris=3 (KONSISTEN)	0

7.2 Percobaan pembanding -- alat harus MAMPU mendeteksi kesalahan

Sesuai ketentuan D.3, syarat kuota pada UPDATE supervisors (method IncrementSupervised) dihapus SEMENTARA -- hanya klausa WHERE-nya -- lalu dijadikan biner server tersendiri dan dijalankan ulang dengan data yang sama; sesudah percobaan, kode dikembalikan persis seperti semula.

Perbedaan yang diberlakukan SEMENTARA

```
// SEBELUM diberlakukan (kode ASLI):
`UPDATE supervisors SET supervised_count = supervised_count + 1
 WHERE id = $1 AND supervised_count < quota RETURNING ` + supervisorColumns,
 supervisorID)

// KONFIGURASI PEMBANDING (syarat kuota dihapus sementara):
`UPDATE supervisors SET supervised_count = supervised_count + 1
 WHERE id = $1 RETURNING ` + supervisorColumns, supervisorID)

// setelah run: git restore app/repository/supervisor_repository.go
// (diperiksa: git diff -- file kosong -- kode utuh kembali)
```

Percobaan pembanding

```
[PEMBANDING] 30 mahasiswa berebut pembimbing #1 secara serentak (tanpa guard)...

=== assign_broken ===
  201                                3
  500 INTERNAL_ERROR                 27
  TOTAL                              30
waktu total : 43.464829ms
db: kuota=3 supervised_count=3 baris=3
deadlocks=0
```

Log server pasca run menjabarkan penyebabnya, persis pola naive pada Bagian A:

Baris ERROR pada log server pembanding (level ERROR, msg request_failed)

```
INTERNAL_ERROR: terjadi kesalahan pada server: menambah bimbingan:
ERROR: new row for relation "supervisors" violates check constraint
"supervisors_supervised_within_quota"   (SQLSTATE 23514)
```

Uji	Hasil 27 mahasiswa ke-4 s/d ke-30	Kesimpulan
Dengan guard (kode asli)	27 x 409 SUPERVISOR_FULL -- ditolak sejak pemeriksaan kuota oleh UPDATE bersyarat	Perilaku yang benar: penolakan kuota terjadi pada layer aplikasi
Tanpa guard (diberlakukan sementara)	27 x 500 INTERNAL_ERROR -- yang menolak justru CHECK constraint database (SQLSTATE 23514)	Uji MAMPU mendeteksi kesalahan: tanpa guard, 27 mahasiswa menembus pemeriksaan dan hanya diselamatkan CHECK constraint dengan kode 500 -- sesuai semangat C.6, pelanggaran constraint TIDAK boleh muncul pada kode yang benar

7.3 Uji tambahan: 20 pasang switch berlawanan arah (bebas deadlock)

Kuota kedua pembimbing dinaikkan sementara menjadi 25 agar seluruh setup sequential (40 assign) dilayani; sesudah run kuota dikembalikan seperti seed (3/5/10) dan data direset. Semua 40 switch berjalan 200 OK, kedua jumlah peserta 20/20 konsisten dengan jumlah barisnya, waktu total jauh di bawah 1 detik, dan tidak ada deadlock yang terdeteksi:

Percobaan switch

```
Setup: mendaftarkan mahasiswa ke pembimbing asal satu per satu...

20 pasang mahasiswa switch pembimbing berlawanan arah secara serentak...

=== switch ===
  200                                40
  TOTAL                              40
waktu total : 324.864887ms
1 kuota=25 supervised_count=20 baris=20
2 kuota=25 supervised_count=20 baris=20
deadlocks=0
```

Uji fungsional satu per satu menutup perilaku yang tidak terungkap oleh uji beban:

Tahap	Skenario	Hasil yang diamati
1	Assign 3 mahasiswa ke pembimbing #1 (kuota 3)	3 x 201
2	Assign mahasiswa ke-4 ke pembimbing #2 (kuota 5)	201
3	Switch mahasiswa tersebut 2 -> 1 saat #1 sudah 3/3	409 SUPERVISOR_FULL; posisi mahasiswa tetap supervisor_id=2 di tabel supervisions (atomik)
4	PATCH kuota #1 dengan version terbaru (1)	200; version pada response naik menjadi 2
5	PATCH kuota #1 lagi dengan version lama (1)	409 VERSION_CONFLICT
6	PATCH kuota #1 = 2 < supervised_count = 3	409 QUOTA_BELOW_SUPERVISED
7	PATCH dengan token role user	403 FORBIDDEN

8. Tugas Mandiri D.4 -- Analisis Singkat

8.1 Mengapa 'satu mahasiswa satu pembimbing' butuh kunci tambahan

UPDATE bersyarat pada IncrementSupervised (app/repository/supervisor_repository.go:144-147) hanya membuat atomik perebutan kursi pada SATU baris pembimbing: pemeriksaan kuota dan penyimpanan terjadi dalam satu pernyataan. Tetapi aturan 'satu mahasiswa satu pembimbing' adalah check-then-act milik mahasiswa YANG SAMA yang dapat menyentuh baris pembimbing BERBEDA: pada Assign (app/service/supervisor_service.go:86-92) selang antara pemanggilan FindByStudentID dan InsertSupervision, dua request serentak mahasiswa yang sama pada isolation READ COMMITTED dapat sama-sama membaca 'belum dibimbing' lalu sama-sama menulis; pada Switch (supervisor_service.go:147-174) verifikasi bimbingan lama lalu DELETE dan INSERT meninggalkan selang yang jauh lebih lebar. Kunci tambahan LockStudent ... FOR UPDATE (supervisor_repository.go:101-112), dipanggil sebagai langkah 1 pada Assign dan Switch (supervisor_service.go:81 dan 137), memakai baris students sebagai gembok: request milik mahasiswa yang sama antre sampai transaksi tertangani selesai, sehingga keputusan yang dibaca masih sah pada saat ditulis. UNIQUE(student_id) pada migration 007 tetap dipertahankan sebagai jaring pengaman terakhir -- bila terpicu, seluruh transaksi dibatalkan bersama kenaikan kuota yang tadi terjadi.

8.2 Form kuota ditinggalkan 30 menit: pessimistic vs optimistic

Bila PATCH kuota memakai pessimistic locking, baris pembimbing harus terkunci selama form berdiri -- sekitar 30 menit. Seluruh POST /supervisors/:id/assign dan POST /supervisions/switch ke dosen itu antre di belakang kunci yang tidak sedang menunggu pekerjaan apa pun, setiap transaksi menahan salah satu dari 10 koneksi pool (DB_MAX_CONNS), dan bila klien terputus transaksi menjadi idle in transaction sampai diakhiri paksa -- satu admin melumpuhkan penetapan untuk dosen itu. Kode saya memakai optimistic locking persis untuk kasus ini (app/repository/supervisor_repository.go:200-219): SELAMA form dibuka tidak ada kunci dipegang; saat admin menekan Simpan, UPDATE hanya sukses bila `version` masih yang terlihat klien (Klausa `WHERE id = $2 AND version = $3 AND $1 >= supervised_count` dan `version = version + 1`). Ketika terjadi tabrakan, UPDATE menyentuh 0 baris, service membedakannya lewat pencarian baris terkini (supervisor_repository.go:211-218), dan klien menerima 409 VERSION_CONFLICT -- dibuktikan pada tahap 5 Uji Fungsional bagian 7.3. Klien tinggal muat ulang lalu simpan ulang, tanpa mahasiswa satu pun tertahan.

8.3 Jika fn di dalam WithTx mengirim email

TxManager adalah satu-satunya pintu transaction dan secara sadar mengulang kegagalan yang dapat diulang: WithTx menjalankan ulang runOnce hingga maxAttempts = 3 bila SQLSTATE 40001 (serialization failure) atau 40P01 (deadlock) muncul, dengan jeda acak (app/repository/tx.go:35-52). Karenanya fn TIDAK punya garansi berjalan sekali: email pemberitahuan ke dosen bisa terkirim dua sampai tiga kali. Kedua, email adalah efek di luar database -- ia tidak ikut dibatalkan saat ROLLBACK, sehingga dosen dapat menerima pemberitahuan atas penetapan yang pada akhirnya batal; dan bila pengiriman SMTP di dalam fn gagal, penetapan yang sebetulnya sah ikut batal. Ketiga, latensi jaringan selama pengiriman membuat transaksi memegang koneksi pool dan kuncinya lebih lama dari seharusnya -- melanggar aturan emas #2 pada A.7. Yang benar: kirim email SETELAH WithTx kembali tanpa error (post-commit), atau bila keandalan harus dijamin, tulis baris notifikasi ke tabel outbox DI DALAM transaction yang sama (ikut commit/rollback), lalu worker di luar transaction mengirim sekali dan menandai terkirim.

9. Rekapitulasi Pemenuhan Tugas Mandiri

Butir	Panggilan ketentuan	Status dan bukti
D.2.1	Migration supervisors (id, name, quota, supervised_count, version) + supervisions (student_id, supervisor_id)	Terpenuhi -- migrations/007_supervisors.sql; UNIQUE(student_id) menjamin satu pembimbing per mahasiswa
D.2.2	POST /supervisors/:id/assign berpermission supervision:manage; UPDATE bersyarat + kunci baris students	Terpenuhi -- supervisor_service.go:58-109; route.go:102-104; kunci di repo.go:101-112
D.2.3	POST /supervisions/switch atomik & bebas deadlock	Terpenuhi -- supervisor_service.go:115-181; LockSupervisors urutan id (repo.go:116-139); bukti bagian 7.3 (40/40 200, 0 deadlock)
D.2.4	PATCH /supervisors/:id optimistic locking	Terpenuhi -- repo.go:200-219 (version = version + 1); bukti tahap 4-5 bagian 7.3
D.2.5	Repository DBTX/WithTx; transaction dibuka lewat TxManager	Terpenuhi -- semua method memakai r.db DBTX (repo.go:37-47); service membuka tx lewat s.tx.WithTx (TxManager, tx.go)
D.3	Keluaran uji beban x3 + percobaan pembanding	Terpenuhi -- bagian 7.1/7.2: 30 assign serentak tepat 3 keberhasilan (3 kali); pembanding tanpa guard: 27 x 500 deteksi kesalahan; kode dikembalikan
D.4	Tiga analisis satu paragraf	Terpenuhi -- bagian 8.1-8.3, masing-masing merujuk baris kode proyek sendiri

Keluaran yang sama juga terekam pada logs/app.log (level WARN request_rejected untuk seluruh 409 dan 403, tanpa 'context deadline exceeded' dan tanpa 'idle in transaction' pada pg_stat_activity) dan pg_stat_database.deadlocks tidak bertambah sama sekali selama seluruh rangkaian pengujian.

Pernyataan: laporan ini disusun berdasarkan percobaan yang dijalankan langsung terhadap kode pada folder latihan-fiber/MODUL_9. Versi kode yang diuji sesuai commit D.2 (d3d6bee). Berkas sumber uji, alat beban (tools/loadtest_supervisors), dan seluruh perubahan terkait diunggah ke repositori.